

Capstone Design Manager

Project Documentation

Capstone Design Manager — Deployment Documentation

Summary of Contents

This document covers everything needed to deploy the Capstone Design Manager application, both locally and to Google Cloud. It is recommended that you read through each section in order, as later sections build on earlier ones.

Section	Description
Required Technologies	Full list of technologies used, broken down by deployment context
Local Deployment	Step-by-step guide for getting the app running on your local machine
Setting Up Auth0	Auth0 tenant creation, trigger configuration, and API setup
Cloud Deployment	GCP project setup, VM provisioning, domain registration, SSL, and deployment scripts
Common Issues	Frequently encountered problems and their solutions

Required Technologies

Basic familiarity with this tech stack is a significant advantage and will help deployment go smoothly.

All Technologies Used

1. Git
2. GitHub
3. Docker
4. Docker Compose
5. Django (within Docker containers)
6. Postgres / psql (within Docker containers)
7. Vue.js (frontend)
8. Auth0
9. Google Cloud (Compute Engine & Domains)
10. Google Cloud CLI
11. Let's Encrypt (deployment scripts)
12. Certbot (deployment scripts)
13. Nginx (Docker container for cloud deployment only)
14. NPM (for serving static files to Nginx in the production environment)
15. Mailhog (optional configuration)
16. Gmail SMTP (optional configuration)

For Local Development and Deployment

1. **Git / GitHub account** — you will need to be able to clone the repository.
2. **Docker** — Docker must be installed and the Docker daemon must be running to launch the app locally. Docker Desktop can be helpful if you are not familiar with the

CLI, though some terminal usage is required.

For Cloud Hosting

1. **Google Cloud account / project setup**
2. **Domain registration through Google**
3. **gcloud CLI** — required for running the deployment scripts.

Necessary for Both Local and Cloud

1. **Auth0 account and application setup** — see the [Setting Up Auth0](#) section.

Local Deployment

Note: It is strongly recommended that you get a local deployment working before attempting to host the app on the cloud.

0. Prerequisites

Ensure the following are installed on your machine before proceeding:

1. **Docker** — Docker Desktop is available at <https://docs.docker.com/desktop/>. Select your operating system and follow the installation instructions.
2. **Git** — check if it is already installed with `git --version`. If not, install it from <https://git-scm.com/book/en/v2/Getting-Started-Installing-Git>.

If you are on Windows, use a Unix-like terminal such as Cygwin for the steps below.

1. Clone the Repository

```
git clone https://github.com/jmburke4/capstone-design-manager.git
cd capstone-design-manager
```

For local development, use the `main` branch. For cloud deployment, see the [Cloud Deployment](#) section, which uses the `Cloud-V2` branch.

2. Set Up Auth0

Before configuring your environment files, you must have an Auth0 tenant set up. The environment files require Auth0 values that are only available after completing the Auth0 setup.

Go to the [Setting Up Auth0](#) section, complete it, then return here.

3. Configure Environment Files

Inside the root of the cloned repository, create your environment files by copying the provided examples:

```
cp .env.example .env.dev
cp .env.example.db .env.dev.db

# Navigate to the frontend directory to create the frontend env file
cd frontend
cp .env.example.local .env.dev.local
cd ..
```

Note: Files beginning with `.` are hidden by default in most file explorers. Refer to your file explorer's documentation to reveal hidden files if needed.

Note: Never commit your development or production environment files. They contain secrets and are excluded from version control for this reason.

Backlog: Consolidating the environment files into a single file is a backlog item. The frontend env file in particular is redundant as it uses the same values as `.env.dev`.

Using a text editor of your choice, open each file and update the values as described in the guides below.

.env.dev value guide

The format is `KEY=VALUE`. The guide below is for reference only and is not syntactically accurate.

--	--

Key	Notes
DEBUG	Set to <code>1</code> for local development. Change to <code>0</code> in production.
SECRET_KEY	Set to any secret key of your choice.
DJANGO_ALLOWED_HOSTS	Leave as-is. For local deployment: <code>localhost 127.0.0.1 [::1] backend</code>
APP_BASE_URL	Set to <code>http://localhost:5173</code> for local deployment of the Cloud-V2 branch. In cloud deployment, change this to your base domain.
AUTH0_DOMAIN	Found in your Auth0 dashboard under Application → Settings → Basic Information.
AUTH0_CLIENT_ID	Found in your Auth0 dashboard under Application → Settings → Basic Information.
AUTH0_AUDIENCE	Set to <code>https://backend-api-capstone/</code> . Configured during Auth0 setup.
SQL_ENGINE	Leave as-is.
SQL_DATABASE	Your desired database name. Must match <code>POSTGRES_DB</code> in <code>.env.dev.db</code> .
SQL_USER	Your desired database username. Must match <code>POSTGRES_USER</code> in <code>.env.dev.db</code> .
SQL_PASSWORD	Your desired database password. Must match <code>POSTGRES_PASSWORD</code> in <code>.env.dev.db</code> .
SQL_HOST	Leave as-is: <code>db</code>
SQL_PORT	Your desired port. <code>5432</code> is the standard Postgres port.
DATABASE	Leave as-is.
MINIO_ROOT_USER	Your desired MinIO username.
MINIO_ROOT_PASSWORD	Your desired MinIO password.
AWS_STORAGE_BUCKET_NAME	Your desired bucket name.

Email values	Delete these if you are not using Mailhog or Gmail SMTP.
--------------	--

.env.dev.db value guide

Key	Notes
POSTGRES_USER	Must match SQL_USER in .env.dev .
POSTGRES_PASSWORD	Must match SQL_PASSWORD in .env.dev .
POSTGRES_DB	Must match SQL_DATABASE in .env.dev .

frontend/.env.dev.local value guide

See [Section 3.1 — Basic Setup](#) if you are unsure where to find these values.

Key	Notes
VITE_AUTH0_DOMAIN	Same value as AUTH0_DOMAIN in .env.dev .
VITE_AUTH0_CLIENT_ID	Same value as AUTH0_CLIENT_ID in .env.dev .
VITE_AUTH0_AUDIENCE	Same value as AUTH0_AUDIENCE in .env.dev .

4. Start the Docker Daemon

Ensure Docker is running before proceeding:

```
# macOS
open -a Docker
```

You can also open Docker Desktop directly. On first launch, you may need to grant Docker system permissions and walk through the installation process.

Verify Docker Compose is available:

```
docker-compose --version
# or
docker compose --version
```

If neither command returns a Docker Compose version, install it from <https://docs.docker.com/compose/install/>.

5. Build and Launch Containers

From the root of the project, run:

```
docker-compose -f docker-compose.dev.yml up --build -d
```

The first build typically takes around one minute. If prompted for permissions, select **Allow**.

Note: If you see a Mailhog error, it can be safely ignored unless you are actively using Mailhog.

After the script finishes, verify that containers are running using `docker ps` or Docker Desktop. If containers are not running, review the error output in Docker Desktop. The most common cause is an incorrectly configured environment file.

6. Run Migrations

```
docker-compose -f docker-compose.dev.yml exec backend python manage.py migr
```

If you have added new models or a new app, run `makemigrations` first:

```
docker-compose -f docker-compose.dev.yml exec backend python manage.py
```

7. Create a Django Superuser

A superuser is required for data import and admin access.

```
docker-compose -f docker-compose.dev.yml exec backend python manage.py crea
```

Answer the prompts. Take note of the credentials — a password manager is recommended.

8. Verify the App

Navigate to <http://localhost:5173> in a browser. You should see the app homepage and be able to sign up as a sponsor using the Auth0 tenant.

9. Verify Django Admin Panel Access

Navigate to <http://localhost:8000/admin/login/?next=/admin/> and log in with the superuser credentials you created.

Note: For cloud deployment, admin panel access is handled differently and requires additional Auth0 configuration. See [Step 13](#) for details.

10. Verify Student Account Creation

To access the frontend as a student user, add an entry in the student table with their crimson.ua.edu email. After that, a student can create an account via: landing page → Sign Up → I am a student → Email/password with their crimson email → verify email → log in.

11. Spin Down Containers

```
docker-compose -f docker-compose.dev.yml down
```

12. Verify Cloud-V2 Branch Locally (Recommended before cloud deployment)

Checkout the `Cloud-V2` branch and spin up containers to verify local functionality of the

cloud branch before attempting a cloud deployment.

```
git checkout Cloud-V2
git branch # Verify you are on Cloud-V2

docker-compose -f docker-compose.dev.yml up --build -d
```

You should not need to reconfigure your environment files. If you run into issues, try building with `--no-cache`.

13. Verify Admin Panel Functionality (Cloud-V2)

Admin access works differently on the Cloud-V2 branch. Rather than using a Django superuser directly, you must create an admin sponsor account through Auth0.

A walkthrough is available in the documentation video starting at approximately the 10:25 mark: <https://www.youtube.com/watch?v=FEcXGf6hiFY&feature=youtu.be>

1. Create a sponsor account using Google SSO.
2. In the Auth0 dashboard, navigate to **User Management → Users**.
3. Select the three dots to the right of the sponsor account and choose **More Actions → Assign Roles → admin**.
4. Click **Assign**.
5. Refresh the sponsor's homepage — the admin panel button should now appear in the sidebar.
6. Click the admin panel button and verify that login to the Django admin panel is successful.

Note: On a local deployment of Cloud-V2, the admin login guard may not behave as expected due to browser caching. This is resolved in the cloud-deployed version.

14. Spin Down Containers

```
docker-compose -f docker-compose.dev.yml down
```

Setting Up Auth0

Read all steps carefully before beginning. Some values must not be changed; others are flexible. Use your best judgment where noted.

1. Create an Account

- Navigate to: <https://auth0.com/signup>
- It is recommended to register with an email address rather than SSO, as it is easier to locate your account later.
- Select **Personal** for account type.
- Leave advanced settings unchecked.

2. Create a New Application

After signing up, you will be prompted to create a new application.

- **Name:** capstone-design-manager
- **Technology:** Vue
- **Application type:** Single Page Application
- Click **Create Application**.

3. First Integration Phase (Local Deployment)

3.1 Basic Setup

- In the Vue Quickstart, set the **Application Origin** field to `http://localhost:5173` and click **Continue**.
- The **Callback URLs**, **Logout URLs**, and **Web Origins** fields should all contain `http://localhost:5173`.
- Click **Update Required Settings**.
- Do not integrate with AI when prompted.
- Navigate to the **Settings** tab (immediately to the right of Quickstart).
- Take note of the **Domain** and **Client ID** fields under Basic Information. Your Auth0 environment values will look something like this:

```
AUTH0_DOMAIN=dev-qjyd077ykn3qqq7v.us.auth0.com
AUTH0_CLIENT_ID=WMPPr5zJLNFI0j9A8iUymDfAsP2mUXsn3
AUTH0_AUDIENCE=https://backend-api-capstone/    # Configured in the next step
```

Do not update your environment files yet — finish configuring Auth0 first.

3.2 Create the Triggers

The Auth0 scripts are stored in the repository at `capstone-design-manager/Auth0-Scripts/`. You can reference them from a local clone or directly from [GitHub](#).

On your Auth0 dashboard, navigate to **Actions → Triggers**.

Pre-user-registration trigger:

1. Under **Sign Up & Login**, select **pre-user-registration**.
2. Click the **Add Action** button (+ icon) in the upper right.
3. Select **Create Custom Action**.
 - **Name:** `ValidateRole&Domain`
 - **Trigger:** Pre User Registration
 - **Runtime:** Node 22
 - Click **Create**.
4. Delete the template code and paste in the contents of `Auth0-Scripts/pre-user-registration.txt`.
5. Click **Deploy**.
6. Click **Back to Triggers**.
7. Drag the `ValidateRole&Domain` action between **Start** and **Complete**.
8. Click **Apply**.

Post-login trigger:

1. Under **Sign Up & Login**, select **post-login**.
2. Click the **Add Action** button (+ icon).
3. Select **Create Custom Action**.
 - **Name:** `AssignRole&EnforceAccess`
 - **Trigger:** Login / Post Login

- **Runtime:** Node 22
 - Click **Create**.
4. Delete the template code and paste in the contents of `Auth0-Scripts/post-login.txt`.
 5. Click **Deploy**.
 6. Click **Back to Triggers**.
 7. Drag the `AssignRole&EnforceAccess` action between **Start** and **Complete**.
 8. Click **Apply**.

3.3 Create the Auth0 Backend API

Step 1 — Create the API:

1. Navigate to **Applications** → **APIs** in the left sidebar.
2. Click **+ Create API**.
 - **Name:** `backend-api`
 - **Identifier:** `https://backend-api-capstone/` — **do not change this value.**
 - Leave all other fields as default.
 - Click **Create**.
3. Navigate to **Application Access**.
 - In the `capstone-design-manager` row, click **Edit**.
 - Under **User Access**, set **Authorization** to **Authorized** and click **Save**.
 - Under **Client Access**, set **Authorization** to **Authorized** and click **Save**.

Step 2 — Create the backend application:

1. Navigate to **Applications** → **Applications** in the left sidebar.
2. Click **+ Create Application**.
 - **Name:** `backend-api`
 - **Type:** Machine to Machine Application
 - Click **Create**.
3. When prompted to select an API, select the `backend-api` with audience `https://backend-api-capstone/`.
4. Click **Authorize**.

3.4 Create the Admin Role

This step is not strictly required for a basic local deployment, but is required for local deployment of the Cloud-V2 branch.

1. Navigate to **User Management → Roles** in the left sidebar.
2. Click **+ Create Role**.
 - **Name:** `admin` — **do not change this value**. The application expects the role to be named exactly `admin`.
 - **Description:** e.g., "Grants access to the Django admin panel on the cloud branch."
 - Click **Save**.

Auth0 Setup for Local Deployment is Now Complete

Return to [Step 3 — Configure Environment Files](#) and update your `.env` files with the Auth0 values you collected.

4. Integrating Cloud Deployment with Your Auth0 Tenant

1. In the Auth0 dashboard, navigate to **Applications → Applications**.
2. Select the `capstone-design-manager` application.
3. Add your production domain to each of the following fields (in addition to the existing `localhost` entries):
 - **Allowed Callback URLs**
 - **Allowed Logout URLs**
 - **Allowed Web Origins**

Each field should look something like:

`http://localhost:5173/, https://www.ua-capstone-projects.com/, https://ua-c`

1. Ensure the `admin` role is created and assigned to any users who need Django admin access.

Cloud Deployment

Note: All notes below should be read before running any scripts.

Note: The cloud configuration lives on the `Cloud-V2` branch, separate from `main`. Consolidating these into `main` is a backlog item. All cloud deployment steps should be performed on `Cloud-V2`.

Note: It is strongly recommended to successfully deploy the app locally before attempting a cloud deployment.

Note: To bring local changes into the cloud deployment: (1) PR changes into `main`, (2) verify they work, (3) create a local backup of `Cloud-V2`, (4) rebase `Cloud-V2` onto `main`, (5) test the rebased version locally.

Complete all prerequisites below before running any scripts.

0. Review `config.sh`

All deployment scripts source `scripts/config.sh` for their variables. If you change any values from the defaults described in this guide, update `config.sh` accordingly before running any scripts.

1. GCP Project Setup

Create the Project

1. Go to: <https://console.cloud.google.com/projectcreate>
2. Fill in the following:
 - **Project name:** Capstone Design App Prod
 - **Project ID:** `capstone-design-app-prod` (referenced in `scripts/config.sh`)
 - **Organization:** No organization
3. Click **Create**.

Enable Billing

1. Go to: <https://console.cloud.google.com/billing/linkedaccount?project=capstone-design-app-prod>
2. Click **Link a billing account**.
3. Select your billing account (eligible for \$300 in free credits).
4. Click **Set Account**.
5. Verify that billing shows as **Active** with a green checkmark.

2. Clone the Repository

```
git clone https://github.com/jmburke4/capstone-design-manager
cd capstone-design-manager
git checkout Cloud-V2
git branch # Verify you are on Cloud-V2
```

3. Install and Configure the gcloud CLI

1. Install the gcloud CLI: <https://cloud.google.com/sdk/docs/install-sdk>
2. Ensure your account is authenticated and configured for the gcloud CLI on your device.

4. SSH Key Setup (Optional)

The deployment scripts SSH into your GCP Compute VM. To avoid being prompted for a passphrase repeatedly, use one of the following options.

Option A — Remove the Passphrase

```
# Remove existing SSH keys
rm ~/.ssh/google_compute_engine ~/.ssh/google_compute_engine.pub

# Clear SSH config
gcloud compute config-ssh --remove
```

The next time you run `gcloud compute ssh`, new keys will be generated. When prompted for a passphrase, press **Enter** to leave it empty.

Option B — Use the SSH Agent

```
# Start the SSH agent
eval $(ssh-agent)

# Add your key and enter your passphrase when prompted
ssh-add ~/.ssh/google_compute_engine
```

5. Authenticate with GCP

```
# Log in to gcloud (opens a browser)
gcloud auth login

# Set the default project
gcloud config set project capstone-design-app-prod

# Verify
gcloud config get-value project
# Expected output: capstone-design-app-prod
```

6. Domain Registration & DNS Configuration

The following steps use the values from the reference deployment. Substitute your own values where appropriate.

6.1 Register the Domain

1. Navigate to the [Google Cloud Domain Registration](#) page.
2. Search for your desired domain (e.g. `ua-capstone-projects.com`) and select it from the results.
3. Fill in contact info and address, then click **Activate**.

6.2 Enable Cloud DNS

1. Enable the Cloud DNS API via the [Cloud DNS metrics page](#).
2. When prompted, select **Use Cloud DNS**.
3. Set visibility to **Limit info available to public**.

6.3 Create DNS Records

Once the domain is registered, create an `A` record pointing to your VM's static IP. Replace `<YOUR_STATIC_IP>` with your actual static IP before running.

```
PROJECT=capstone-design-app-prod
ZONE_NAME=ua-capstone-projects-com
VM_IP=<YOUR_STATIC_IP>

gcloud dns record-sets transaction start \
  --zone="$ZONE_NAME" \
  --project="$PROJECT"

gcloud dns record-sets transaction add "$VM_IP" \
  --name="www.ua-capstone-projects.com." \
  --ttl=300 \
  --type=A \
  --zone="$ZONE_NAME" \
  --project="$PROJECT"

gcloud dns record-sets transaction execute \
  --zone="$ZONE_NAME" \
  --project="$PROJECT"
```

Note: DNS propagation can take up to 48 hours, though it typically resolves within a few minutes on Google Cloud DNS.

7. Run the Deployment Scripts

The scripts must be run in the following order:

Step	Script	Description
		Configure all deployment variables. Review before running

1	<code>config.sh</code>	anything.
2	<code>01-create-vm.sh</code>	Validates the GCP environment, creates firewall rules, and provisions the VM.
3	<code>02-setup-vm.sh</code>	Configures the VM: system updates, prerequisite installs, rootless Docker, port forwarding, and repo clone. Takes a few minutes.
4	<code>generate-secrets.sh</code>	Generates production <code>.env</code> files and transfers them to the VM.
5	<code>03-deploy-app.sh</code>	Builds the frontend, builds Docker images, starts containers, runs migrations and <code>collectstatic</code> . This is the longest-running script (~5+ minutes) and has historically been the most error-prone. After this step, create a Django superuser.
6	<code>04-setup-ssl.sh</code>	Obtains a TLS certificate via Certbot and Let's Encrypt, and configures Nginx to serve HTTPS traffic.

8. PLEASE NOTE DIFFERENCES

There are a few key differences between the structure of the cloud deployment and that of the local deployment.

1. In the cloud/production version, there is no frontend container
2. In the cloud/production version, we use Nginx to serve through HTTPS
3. Because of Nginx, we must run `npm run build` inside the frontend directory of the VM in order for Nginx to serve our frontend.
4. The production version uses different Docker, docker compose and environment files (`.env` files vary in name and value composition only. The structure remains the same).
5. Currently, these cloud/production version differences are only on the `Cloud-V2` branch, and have yet to be consolidated into main.

Script Details

`01-create-vm.sh` — Validates that the following are all true before proceeding: the gcloud CLI is authenticated, the project exists as defined in `config.sh`, billing is enabled, Compute Engine is enabled, and the `Cloud-V2` branch exists in the repository.

It then sets the project, ensures the VM's external IP is static, creates the necessary firewall rules, and initializes the VM if it does not already exist.

02-setup-vm.sh — Performs system updates, installs prerequisites, configures rootless Docker, sets up iptables and port forwarding, and clones the repo into `$HOME/capstone`. Multiple verification steps are included for debugging purposes and may be cleaned up in future versions.

generate-secrets.sh — Creates the production environment files and sends them to the VM. Output is printed to the terminal for debugging purposes. Several redundant checks are included that may be cleaned up in future versions.

03-deploy-app.sh — Copies the `.env` files from `generate-secrets.sh`, builds the frontend for the Nginx container, builds Docker images, starts containers, and runs migrations and `collectstatic`. Also includes several verification steps. This script is the most error-prone and the slowest to run.

04-setup-ssl.sh — Runs verification steps, uses Certbot (webroot mode) to obtain a TLS certificate from Let's Encrypt, and configures Nginx to serve HTTPS traffic. This is the second most error-prone script. Inline comments in the file provide additional

[Project Video](#)

[Backlog](#)

[Cyber Security](#)

[Features](#)

[Deployment](#)

[User Guide](#)

[Development](#)

[FAQs](#)

[Testing](#)

[← Home](#)

[Documentation](#)

[Authors](#)

Issues

Prompts — Depending on your gcloud setup, you may be prompted to type a passphrase multiple times throughout the deployment scripts, as many times as the VM. See [Section 4 — SSH Key Setup](#) to reduce this friction.

Issues — Many potential issues are tangential to the deployment scripts and are related to environment configuration, gcloud setup, Auth0 settings, and environment variables. These are common sources of bugs.

Script failures — If a script hangs or always crashes at a certain point, SSH into the VM and run the failing commands manually for more detailed error messages.

```
ssh capstone-prod-vm --zone=us-central1-b --project=capstone-
```

SSL setup timing — Sometimes simply waiting a few minutes after the initial VM setup resolves SSL issues. Wait before running `04-setup-ssl.sh` .

DNS caching — If the SSL script reports that your domain resolves to the wrong IP, you may need to flush your local DNS cache:

```
# macOS
sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder
```

The warning will look like:

```
⚠ WARNING: DNS for ua-capstone-projects.com resolves to 208.91.112.55 (expected 104.133.211.10)
Continue anyway? (yes/no):
```

Nginx infinite restart loop — Ensure that only one `.conf` file exists in `nginx/conf.d/` at a time. At the end of the deployment process, `default.conf` and `redirect.conf` are renamed to `.bak` after `https.conf` is created. If both `https.conf` and one of the other `.conf` files coexist, Nginx will have conflicting backend references and restart indefinitely.

Nginx update issues - If a frontend update is not apply, ensure that you've re-run `npm run build` inside the frontend directory of the VM. You may also need to run `collectstatic` to get Django admin panel CSS styling working properly (covered in `03-deploy-app.sh`).

Backend container issues — The vast majority of backend container issues are caused by either unapplied migrations or misconfigured `DJANGO_ALLOWED_HOSTS` / `backend/core/settings.py` values.

Auth0 issues — Ensure the Auth0 setup was followed carefully. Most issues are caused by missing production domains in the **Allowed Callback URLs**, **Allowed Logout URLs**, and **Allowed Web Origins** fields in the application settings.

Environment file issues — Carefully review all `.env` files and ensure they are all present and correctly configured.

- **Local deployment:** `.env.dev` , `.env.dev.db` , `frontend/.env.dev.local`
- **Production:** Same files with `dev` replaced by `production` (except the frontend file, which should be named `.env.production`)

Backlog: Consolidating the `.env` files is a known backlog item.

Docker issues — Ensure the Docker daemon is running. Clearing the cache, containers, and volumes can resolve persistent issues, but note that this deletes data. This is acceptable during development but should be approached carefully in production.

Helpful Commands

Git

All standard Git commands apply during development.

Docker

```
# Bring containers down and remove volumes (deletes data — use with caution)
docker compose -f <compose script> down -v
```

```
# Check the status of running containers
docker ps
```

```
# View recent logs from a specific container
docker logs <container-name> --tail 50
```

gcloud CLI

```
# SSH into the production VM
gcloud compute ssh capstone-prod-vm \
  --zone=us-central1-b \
  --project=capstone-design-app-prod \
  --tunnel-through-iap
```

```
# List all firewall rules
gcloud compute firewall-rules list
```

```
# Completely delete the VM and all its disks
gcloud compute instances delete capstone-prod-vm \
  --zone=us-central1-b \
  --project=capstone-design-app-prod \
  --delete-disks=all \
```

